

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 5

**Duration : 1 Hour
Total Marks:50**

Annexure A

Analytical Problem Solving

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Step 1 – Problem formulation

- **State:** (x, y) where x = water in the 4-gallon jug ($0 \leq x \leq 4$) and y = water in the 3-gallon jug ($0 \leq y \leq 3$).
- **Initial state:** $(0, 0)$
- **Goal state:** $(2, n)$ for any n – i.e., 2 gallons in the 4-gallon jug.
- **Path cost:** 1 per action (we want the fewest steps).

Step 2 – Production rules (actions)

No.	Condition	Action	Resulting state
1	$x < 4$	Fill the 4-gal jug from pump	$(4, y)$
2	$y < 3$	Fill the 3-gal jug from pump	$(x, 3)$
3	$x > 0$	Empty the 4-gal jug on ground	$(0, y)$
4	$y > 0$	Empty the 3-gal jug on ground	$(x, 0)$
5	$x + y \geq 4, y > 0$	Pour 3-gal into 4-gal until 4-gal is full	$(4, y - (4 - x))$
6	$x + y \geq 3, x > 0$	Pour 4-gal into 3-gal until 3-gal is full	$(x - (3 - y), 3)$
7	$x + y \leq 4, y > 0$	Pour all of 3-gal into 4-gal	$(x + y, 0)$
8	$x + y \leq 3, x > 0$	Pour all of 4-gal into 3-gal	$(0, x + y)$

Step 3 – Solution path (found by BFS – shortest, 6 steps)

Step	State (x, y)	Action	Rule used
0	$(0, 0)$	Start – both jugs empty	–
1	$(0, 3)$	Fill the 3-gallon jug	2
2	$(3, 0)$	Pour 3-gal jug entirely into 4-gal jug	7
3	$(3, 3)$	Fill the 3-gallon jug again	2
4	$(4, 2)$	Pour from 3-gal into 4-gal until 4-gal is full (only 1 gallon fits, 2 remain)	5
5	$(0, 2)$	Empty the 4-gallon jug on the ground	3
6	$(2, 0)$	Pour the 2 gallons from the 3-gal jug into the 4-gal jug	7

Result: The goal state (2, 0) is reached – the 4-gallon jug contains exactly 2 gallons. State-space path: (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0).

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

- i. What are the percept's for this agent?
- ii. Characterize the operating environment.
- iii. What are the actions the agent can take?
- iv. How can one evaluate the performance of the agent?
- v. What sort of agent architecture do you think is most suitable for this agent? Why?

(1) Percepts

- Camera images (panoramic and close-up) of terrain, rocks and soil.
- Spectrometer / chemical-analysis readings of the samples (minerals, water, organics).
- Proximity, LIDAR and tilt/slope sensor readings (obstacles, cliffs, craters).
- Position and motion data: GPS-like navigation, IMU, wheel encoders / odometer.
- Internal status: battery level, solar-panel output, temperature, wheel/motor health.
- Environmental data: ambient temperature, wind, dust and radiation levels.
- Commands and updates uplinked from mission control on Earth.

(2) Characteristics of the operating environment

Property	Value	Reason
Observable	Partially observable	Sensors see only nearby terrain; sub-surface is hidden.
Agents	Single agent	Mostly the rover alone (Earth control only sends commands).
Deterministic?	Stochastic	Wheel slip, dust, unknown soil – outcomes are uncertain.
Episodic?	Sequential	Current move affects energy, position and later tasks.
Static?	Dynamic	Dust storms, temperature and light change while the rover decides.
Discrete?	Continuous	Terrain, position, time and sensor values are continuous.
Known?	Largely unknown	Terrain and sample properties are not known in advance.

Additionally, there is a large communication delay (minutes) with Earth, so the rover needs local autonomy.

(3) Actions

- Move forward / backward, turn left / right, stop, climb, change speed.
- Avoid or go around obstacles; choose a new path.
- Take photographs; point the camera or mast.

- Extend robotic arm, scoop soil, drill into rock, collect and store samples.
- Run on-board spectrometer / chemical analysis on a sample.
- Transmit data to Earth / orbiter; receive commands.
- Recharge (orient solar panels), enter sleep or safe mode, heat itself.

(4) Performance measure

- Number and scientific value / quality of samples collected and analysed.
- Area of Mars surface explored and the number of new sites reached.
- Amount and quality of data successfully transmitted to Earth.
- Safety and survival – no falls, collisions, getting stuck or hardware damage.
- Energy and time efficiency (science per unit energy, mission lifetime).

(5) Suitable agent architecture

A utility-based (goal-based, model-based) learning agent is the most suitable.

- Model-based: the environment is partially observable, so the rover must keep an internal map/state of terrain, battery and position.
- Goal-based: it has explicit goals (reach site X, collect sample Y) and must plan sequences of actions.
- Utility-based: it must trade off conflicting objectives – science value vs. energy use vs. risk of damage – so a utility function picks the best action.
- Learning element: unknown terrain means the agent improves its terrain models and navigation from experience.
- A simple reflex agent is not enough, because reacting only to the current percept cannot handle planning, delay and uncertainty. Some reflex behaviour (emergency stop at a cliff edge) is kept as a safety layer.

3. **Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.**

Step 1 – Problem components (incremental formulation)

Component	Definition
States	Any arrangement of 0 to 8 queens on the board.
Initial state	Empty board – no queens placed.
Actions	Add a queen to a square (in the leftmost empty column) so that it is not attacked by any queen already placed.
Transition model	Returns the board with the new queen added to the chosen square.
Goal test	8 queens are on the board and none is attacked (no two share a row, column or diagonal).
Path cost	Not relevant – only the final configuration matters (each action can be given cost 1).

Why the given figure fails: all 8 queens are placed, but the goal test fails because the queens in column 1 and column 8 attack each other along a diagonal ($|row_1 - row_8| = |1 - 8|$). Hence it is a complete-state configuration that is not a goal state.

Step 2 – Size of the search space

Formulation	Number of states
Naïve: place 8 queens on any squares ($64 \times 63 \times \dots \times 57$)	$\approx 1.8 \times 10^{14}$
One queen per column, any row (8^8)	16,777,216
Incremental, non-attacking queens only (queens added in leftmost empty column)	2,057

The last formulation prunes illegal states early, which makes search practical.

Step 3 – Exploring with search strategies

- **Breadth-First Search:** expands all states level by level (level $k = k$ queens placed). It is complete but needs a lot of memory because many partial boards are stored.
- **Depth-First Search / Backtracking:** places a queen in column 1, then column 2 and so on; if a column has no safe row, it backtracks to the previous column and tries the next row. Memory use is small (only the current path) – this is the usual method.
- **Complete-state formulation + local search (e.g., hill climbing, min-conflicts):** start with all 8 queens on the board (one per column) and move a queen within its column to reduce the number of attacking pairs.

DFS / backtracking finds the first solution (row for columns 1–8) = (1, 5, 8, 6, 3, 7, 2, 4):

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Type of agent

A goal-based problem-solving agent (with a utility-based element for choosing the cab type).

- **Goal:** reach the destination (Goal state: customer at the destination location).
- **Initial state:** customer at the pick-up location.
- **Actions:** select cab type, confirm booking, ride along a road segment to the next location.
- **Transition model:** location(s) after a road segment or booking choice.
- **Path cost:** fare / distance / time, weighted by the comfort preference of the customer.
- The agent formulates the goal, formulates the problem, searches for a sequence of actions (route + cab choice), and then executes it. The environment is observable, deterministic (approximately), sequential and discrete from the app's viewpoint; the cab-type choice uses a utility function (comfort vs. cost vs. waiting time).

Pseudocode

```
function OLA-PROBLEM-SOLVING-AGENT(percept) returns an action
```

```

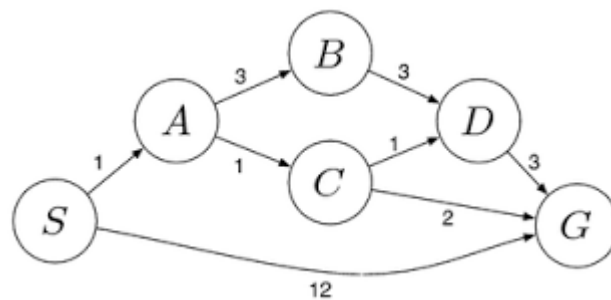
persistent: seq // action sequence, initially empty
            state // current world state
            goal // destination (initially null)
            problem // problem formulation

state <- UPDATE-STATE(state, percept) // pick-up, traffic
if seq is empty then
  goal <- FORMULATE-GOAL(state) // reach destination
  problem <- FORMULATE-PROBLEM(state, goal)
  cabs <- AVAILABLE-CABS(state) // mini, sedan, prime..
  best <- null ; bestUtility <- -infinity
  for each cab in cabs do
    route <- SEARCH(problem, cab) // A*/UCS on roads
    if route is not failure then
      U <- UTILITY(cab, route, comfort_pref)
      if U > bestUtility then
        best <- (cab, route) ; bestUtility <- U
  if best = null then return NO-CAB-AVAILABLE
  seq <- [SELECT(best.cab), CONFIRM-BOOKING] + best.route
action <- FIRST(seq)
seq <- REST(seq)
return action

```

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Edges (directed): $S \rightarrow A = 1$, $S \rightarrow G = 12$, $A \rightarrow B = 3$, $A \rightarrow C = 1$, $B \rightarrow D = 3$, $C \rightarrow D = 1$, $C \rightarrow G = 2$, $D \rightarrow G = 3$.

Method

- UCS keeps a priority queue (frontier) ordered by path cost $g(n)$ from S and always expands the lowest-cost node.
- The goal test is applied when a node is selected for expansion (not when it is generated), which guarantees optimality.
- If a cheaper path to a node already in the frontier is found, its cost is updated.

Step-by-step execution

Step	Node expanded (g)	Successors generated	Frontier (node : cost) after step
1	S (0)	$A = 0+1 = 1$; $G = 0+12 = 12$	A:1, G:12
2	A (1)	$B = 1+3 = 4$; $C = 1+1 = 2$	C:2, B:4, G:12
3	C (2)	$D = 2+1 = 3$; $G = 2+2 = 4$ (better than 12, update)	D:3, B:4, G:4
4	D (3)	$G = 3+3 = 6$ (not better than 4, discard)	B:4, G:4
5	B (4)	$D = 4+3 = 7$ (not better than 3, discard)	G:4
6	G (4)	Goal test passed – stop	–

In step 5, B and G both have cost 4; B was generated first, so it is expanded first (FIFO tie-break). Expanding G first gives the same answer.

Order of expansion: $S \rightarrow A \rightarrow C \rightarrow D \rightarrow B \rightarrow G$

Least-cost path: $S \rightarrow A \rightarrow C \rightarrow G$

Total cost: $1 + 1 + 2 = 4$ cost units.

Interpretation

- The direct route $S \rightarrow G$ costs 12, and $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$ costs 7 – both are worse than $S \rightarrow A \rightarrow C \rightarrow G$ at 4. UCS finds the minimum-cost route because it expands by cost, not by number of hops or depth.
- Paths to G compared: $S \rightarrow G = 12$, $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = 7$, $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G = 10$, $S \rightarrow A \rightarrow C \rightarrow G = 4$ (optimal).
- UCS is complete (when every edge cost $\geq \epsilon > 0$) and optimal. Time and space complexity are $O(b^{(1 + \lceil C^*/\epsilon \rceil)})$, where C^* is the optimal cost. With equal step costs it behaves like breadth-first search.
- For the logistics company, the recommended delivery route is $S \rightarrow A \rightarrow C \rightarrow G$ with the minimum combined fuel-and-time cost of 4.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem